

# 第七章节 自动化测试常用函数

## 本节重点

- 元素定位
- 操作测试对象
- 窗口
- 等待
- 导航
- 弹窗
- 文件上传
- 浏览器参数

## 1.元素的定位

web自动化测试的操作核心是能够找到页面对应的元素，然后才能对元素进行具体的操作。

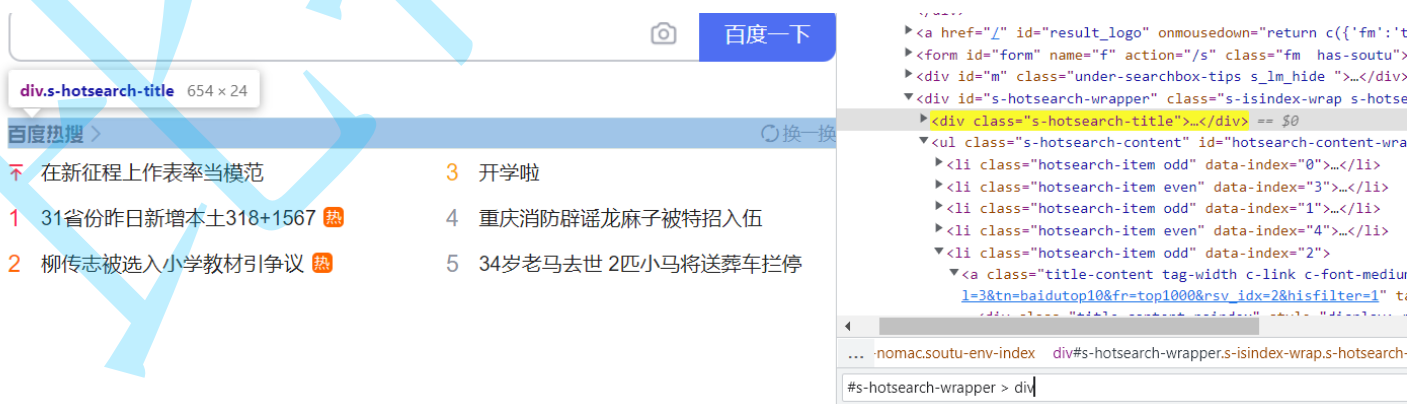
常见的元素定位方式非常多，如id, classname, tagname, xpath, cssSelector

常用的主要由cssSelector和xpath

### 1.1 cssSelector

选择器的功能：选中页面中指定的标签元素

选择器的种类分为基础选择器和复合选择器，常见的元素定位方式可以通过id选择器和子类选择器来进行定位。



定位百度首页的“百度热搜”元素，可以使用通过id选择器和子类选择器进行定位：#s-hotsearch-wrapper > div

“搜索输入框元素”：#kw

“百度一下按钮”：#su

## 1.2 xpath

XML路径语言，不仅可以在XML文件中查找信息，还可以在HTML中选取节点。

xpath使用路径表达式来选择xml文档中的节点

xpath语法中：

### 1.2.1 获取HTML页面所有的节点

`//*`

### 1.2.2 获取HTML页面指定的节点

`//[指定节点]`

`//ul`：获取HTML页面所有的ul节点

`//input`：获取HTML页面所有的input节点

### 1.2.3 获取一个节点中的直接子节点

`/`

`//span/input`

### 1.2.4 获取一个节点的父节点

`..`

`//input/..` 获取input节点的父节点

### 1.2.5 实现节点属性的匹配

`[@...]`

`//*[@id='kw']` 匹配HTML页面中id属性为kw的节点

### 1.2.6 使用指定索引的方式获取对应的节点内容

注意：xpath的索引是从1开始的。

百度首页通过：`//div/ul/li[3]` 定位到第三个百度热搜标签

更便捷的生成selector/xpath的方式：右键选择复制"Copy selector/xpath"

注意：元素的定位方法必须唯一。

案例：如果想要匹配到百度首页指定的新闻文本或者节点集：，直接使用 `#hotsearch-content-wrapper > li` 能够满足吗？

问题：既然可以手动复制 `selector/xpath` 的方式，为什么还有了解语法？

手动复制的selector/xpath表达式并不一定满足上面的唯一性的要求，有时候也需要手动的进行修改表达式

案例：百度首页（需要登陆百度账号）右侧的热搜，复制li标签下的a标签，复制好的的selector为：`#title-content`，xpath为：`//*[@id="title-content"]`，同学们可以手动操作一下，手动复制的表达式是否唯一呢？

## 2.操作测试对象

获取到了页面的元素之后，接下来就是要对元素进行操作了。常见的操作有点击、提交、输入、清除、获取文本。

### 2.1 点击/提交对象

`click()`

```
1 //找到百度一下按钮并点击
2 driver.findElement(By.cssSelector("#su")).click();
```

### 2.2 模拟按键输入

`sendKeys("")`

```
1 driver.findElement(By.cssSelector("#kw")).sendKeys("输入文字");
```

### 2.3 清除文本内容

输入文本后又想换一个新的关键词，这里就需要用到 `clear()`

```
1 driver.findElement(By.cssSelector("#kw")).sendKeys("我爱游戏");
2 driver.findElement(By.cssSelector("#kw")).clear();
3 driver.findElement(By.cssSelector("#kw")).sendKeys("我爱学习");
```

### 2.4 获取文本信息

如果判断获取到的元素对应的文本是否符合预期呢？获取元素对应的文本并打印一下~~

获取文本信息：`getText()`

```
1 String bdtext = driver.findElement(By.xpath("//*[@id="title-
```

```
content"]/span[1]").getText();  
2 System.out.println("打印的内容是: "+bdttext);
```

问题：是否可以通过 `getText()` 获取到“百度一下按钮”上的文字“百度一下”呢？尝试一下  
注意：文本和属性值不要混淆了。获取属性值需要使用方法 `getAttribute("属性名称")`；

## 2.5 获取当前页面标题

```
getTitle()
```

## 2.6 获取当前页面URL

```
getCurrentUrl()
```

# 3. 窗口

打开一个新的页面之后获取到的title和URL仍然还是前一个页面的？

当我们手工测试的时候，我们可以通过眼睛来判断当前的窗口是什么，但对于程序来说它是不知道当前最新的窗口应该是哪一个。对于程序来说它怎么来识别每一个窗口呢？每个浏览器窗口都有一个唯一的属性句柄（handle）来表示，我们就可以通过句柄来切换

## 3.1 切换窗口

1) 获取当前页面句柄：

```
driver.getWindowHandle();
```

3) 获取所有页面句柄：

```
driver.getWindowHandles()
```

3) 切换当前句柄为最新页面：

```
1 String curWindow = driver.getWindowHandle();  
2 Set<String> allWindow = driver.getWindowHandles();  
3 for( String w : allWindow){  
4     if(w!=curWindow){  
5         driver.switchTo().window(w);  
6     }  
7 }
```

注意：执行了 `driver.close()` 之前需要切换到未被关闭的窗口

## 3.2 窗口设置大小

1) 窗口的大小设置

```
1 //窗口最大化
2 driver.manage().window().maximize();
3 //窗口最小化
4 driver.manage().window().minimize();
5 //全屏窗口
6 driver.manage().window().fullscreen();
7 //手动设置窗口大小
8 driver.manage().window().setSize(new Dimension(1024, 768));
```

### 3.3 窗口切换

去掉等待后，获取跳转后的页面元素失败

```
1 //获取所有句柄
2 //获取当前停留页面句柄
3 String curWindow = driver.getWindowHandle();
4 Set<String> allWindow = driver.getWindowHandles();
5 for( String w : allWindow){
6     if(w!=curWindow){
7         driver.switchTo().window(w);
8     }
9 }
```

### 3.4 屏幕截图

我们的自动化脚本一般部署在机器上自动的去运行，如果出现了报错，我们是不知道的，可以通过抓拍来记录当时的错误场景

屏幕截图方法需要额外导入包：

```
1 <dependency>
2     <groupId>commons-io</groupId>
3     <artifactId>commons-io</artifactId>
4     <version>2.6</version>
5 </dependency>
```

```
1 File file = ((TakesScreenshot)webDriver).getScreenshotAs(OutputType.FILE);
2 FileUtils.copyFile(file,new File(filename));
```

代码演示

```
1 //简单版本
2 File srcfile = driver.getScreenshotAs(OutputType.FILE);
3 FileUtils.copyFile(srcfile,new File("my.png"));
4
5 //高阶版本
6 List<String> times = getTime();
7 //生成的文件夹路径./src/test/autotest-2022-08-01/goodsbroser-20220801-214130.png
8 String filename = "./src/test/autotest-"+times.get(0)+"-"+str+"-
  "+times.get(1)+".png";
9 File srcfile = driver.getScreenshotAs(OutputType.FILE);
10 //把屏幕截图放到指定的路径下
11 FileUtils.copyFile(srcfile,new File(filename));
```

### 3.5关闭窗口

```
1 driver.close();
2 注意：窗口关闭后driver要重新定义
```

## 4、等待

通常代码执行的速度比页面渲染的速度要快，如果避免因为渲染过慢出现的自动化误报的问题呢？可以使用selenium中提供的三种等待方法：

### 4.1 强制等待

`Thread.sleep()`

优点：使用简单，调试的时候比较有效

缺点：影响运行效率，浪费大量的时间

### 4.2 隐式等待

隐式等待是一种智能等待，他可以规定在查找元素时，在指定时间内不断查找元素。如果找到则代码继续执行，直到超时没找到元素才会报错。

`implicitlyWait()` 参数：Duration类中提供的毫秒、秒、分钟等方法

示例：

```
1 //隐式等待1000毫秒
2 driver.manage().timeouts().implicitlyWait(Duration.ofMillis(1000));
3 //隐式等待5秒
4 driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
```

隐式等待作用域是整个脚本的所有元素。即只要driver对象没有被释放掉（driver.quit()），隐式等待就一直生效。

优点：智能等待，作用于全局

## 4.3 显示等待

显示等待也是一种智能等待，在指定超时时间范围内只要满足操作的条件就会继续执行后续代码

```
new WebDriverWait(driver, Duration.ofSeconds(3)).until($express)
```

\$press：涉及到selenium.support.ui.ExpectedConditions包下的ExpectedConditions类

返回值：boolean

示例：

```
1 WebDriverWait foo = new WebDriverWait(driver, Duration.ofSeconds(3))
2 foo.until(ExpectedConditions.elementToBeClickable(By.cssSelector("#id")));
```

ExpectedConditions预定义方法的一些示例：

- `elementToBeClickable(By locator)` - 用于检查元素的期望是可见的并已启用，以便您可以单击它。
- `textToBe(Bylocator, String str)` - 检查元素。
- `presenceOfElementLocated(Bylocator)` - 检查页面的 DOM 上是否存在元素。
- `urlToBe(java.lang.String url)` - 检查当前页面的 URL 是一个特定的 URL。

```
1 WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
2 boolean ispass = wait.until(ExpectedConditions.textToBe(By.cssSelector("#s-top-left > a:nth-child(1)"), "新闻"));
3 if(ispass){
4     System.out.println("测试通过");
5 }else {
6     System.out.println("测试失败");
7 }
```

优点：显示等待是智能等待，可以自定义显示等待的条件，操作灵活

缺点：写法复杂

隐式等待和显示等待一起使用效果如何呢？

测试一下

```
1 //隐式等待设置为5s, 显示等待设置为10s, 那么结果会是5+10=15s吗?
2 SimpleDateFormat sim =new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
3 System.out.println(sim.format(System.currentTimeMillis()));
4 driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
5 driver.findElement(By.cssSelector("#hotsearch-content-wrapper > li:nth-child(1) > a > span.title-content"));
6 WebDriverWait wait = new WebDriverWait(driver,Duration.ofSeconds(10));
7 try{
8
9     wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("#hotsearch-content-wrapper > li:nth-child(1) > a > span.title-content")));
10 }catch (Exception e){
11     System.out.println("noelement!");
12 }
13 System.out.println(sim.format(System.currentTimeMillis()));
```

结果：重试多次，最终打印的等待时间有10s、11s....

结论：不要混合隐式和显式等待，可能会导致不可预测的等待时间。

## 5.浏览器导航

常见操作：

### 1) 打开网站

```
1 // 更长的方法
2 driver.navigate().to("https://selenium.dev");
3 // 简洁的方法
4 driver.get("https://selenium.dev");
```

### 2) 浏览器的前进、后退、刷新

```
1 driver.navigate().back();
2 driver.navigate().forward();
3 driver.navigate().refresh();
```

案例：百度首页测试<https://tool.lu/>标签入口

## 6. 弹窗



弹窗是在页面是找不到任何元素的，这种情况怎么处理？使用selenium提供的Alert接口

## 6.1 警告弹窗+确认弹窗



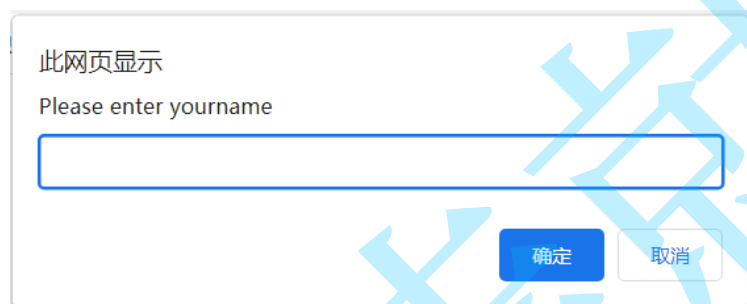
警告弹窗



确认弹窗

```
1 Alert alert = driver.switchTo.alert();
2 //确认
3 alert.accept()
4 //取消
5 alert.dismiss()
```

## 6.2 提示弹窗



```
1 Alert alert = driver.switchTo.alert();
2 alert.sendKeys("hello");
3 alert.accept();
4 alert.dismiss();
```

## 7. 文件上传

点击文件上传的场景下会弹窗系统窗口，进行文件的选择。

selenium无法识别非web的控件，上传文件窗口为系统自带，无法识别窗口元素

但是可以使用sendKeys来上传指定路径的文件，达到的效果是一样的

```
1 WebElement ele = driver.findElement(By.cssSelector("body > div > div >
  input[type=file]"));
2 ele.sendKeys("D:\\selenium2html\\selenium2html\\upload.html");
```

## 9、浏览器参数设置

### 1)设置无头模式

```
ChromeOptions options = new ChromeOptions();
options.addArguments("-headless");
ChromeDriver driver = new ChromeDriver(options);
driver.get("https://www.baidu.com");
driver.quit();
```

### 2)设置浏览器加载策略

```
options.setPageLoadStrategy(PageLoadStrategy.NONE);
```